950 Q&A Series • Section 2 of 10

React + Redux

200 Interview Questions & Answers

useState / useEffect / useMemo / useCallback

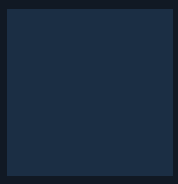
Custom Hooks • Context API • useReducer

React.memo • Fiber • Reconciliation

Redux Toolkit • createSlice • RTK Query

CSR / SSR / SSG • Server Components

Performance • Patterns • Architecture



→ Every question answered with real code examples ←

■ Save • ■ Comment 'JS950' for full PDF • ■■ Repost

Kaushal Singh

Full-Stack Developer | MERN + React Native | Built Fintech Apps at Kanmotech Pvt. Ltd.

1M+ LinkedIn Impressions | 685K in 90 days | Daily Interview Prep content

■■ React + Redux section | 200 Q&A with code | Section 2/10

Q1 What is React? Why use it?

■ Answer

React is a JavaScript UI library by Meta for building component-based interfaces.

Core pillars:

- Declarative — describe WHAT, React handles HOW
- Component-based — reusable, self-contained UI pieces
- Virtual DOM — efficient DOM updates via reconciliation
- Unidirectional data flow — state flows down, events bubble up

```
// Declarative: describe what UI looks like
function Greeting({ name }) {
  return <h1>Hello, {name}!</h1>  // React keeps DOM in sync
}
// Usage:
ReactDOM.createRoot(document.getElementById('root'))
  .render(<Greeting name='Kaushal' />)
```

■ **Interview Tip:**
React is a library (not framework) — it only handles the View layer. You choose your own routing (React Router), state (Redux/Zustand), and fetching (React Query).

Q2 What is a SPA (Single Page Application)?

■ Answer

A SPA loads ONE HTML page and dynamically updates content using JS without full page reloads.

Traditional MPA vs SPA:

- MPA: every navigation = new HTML from server (full reload)
- SPA: JS intercepts navigation, renders components, updates URL

Pros of SPA:

- Faster after initial load (no full reloads)
- Rich interactive UI, app-like feel

Cons of SPA:

- Slower initial load (JS bundle must download first)
- Poor SEO by default (empty HTML shell)
- Complexity: need client-side routing, state management

```
// React SPA — index.html has just this:
<div id='root'></div>

// React takes over from here:
import { BrowserRouter, Routes, Route } from 'react-router-dom'

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path='/' element={<Home />} />
        <Route path='/users' element={<Users />} />
        <Route path='/users/:id' element={<UserDetail />} />
      </Routes>
    </BrowserRouter>
  )
}
// URL changes don't reload page — React Router handles it
```

■ **Interview Tip:**
Next.js and Remix solve SPA's SEO problem by adding Server-Side Rendering. Most production React apps use Next.js for this reason.

Q3 Functional vs Class Components

■ Answer

Modern React uses functional components exclusively. Class components are legacy.

Functional Components (modern):

- Plain JS functions returning JSX
- Use hooks for state and lifecycle
- Less boilerplate, easier to test
- Better performance (no class overhead)

Class Components (legacy):

- ES6 class extending React.Component
- State in this.state, update with this.setState
- Lifecycle: componentDidMount, componentDidUpdate, componentWillUnmount
- More verbose, 'this' binding issues

Only use class components for:

- Error boundaries (no hook equivalent yet!)

```jsx
// Functional Component (USE THIS)
function Counter({ initialCount = 0 }) {
  const [count, setCount] = useState(initialCount)
  useEffect(() => { document.title = `Count: ${count}` }, [count])
  return <button onClick={() => setCount(c => c+1)}>{count}</button>
}

// Class Component (legacy, avoid)
class Counter extends React.Component {
  state = { count: this.props.initialCount ?? 0 }
  componentDidUpdate() { document.title = `Count: ${this.state.count}` }
  render() {
    return <button onClick={() => this.setState(s => ({ count: s.count+1 }))}>{this.state.count}</
  }
}
```

■ Interview Tip:
In 2024+ interviews, you should know class components for code-reading purposes, but always write functional components. The only exception: Error Boundaries still require class components.

Q4 What is JSX? How does it compile?

■ Answer

JSX is syntax sugar that lets you write HTML-like code in JavaScript.
Babel compiles JSX → React.createElement() calls.

JSX Rules:

- One root element (or wrap in <>Fragment</>)
- className instead of class
- htmlFor instead of for
- Expressions inside {} curly braces
- camelCase attributes: onClick, onChange, tabIndex
- Self-close void elements:
 <img />

```
// You write JSX:
const el = <button className='btn' onClick={handleClick}>Click {count}</button>

// Babel compiles to:
const el = React.createElement('button',
  { className: 'btn', onClick: handleClick },
  'Click ', count
)

// Fragments avoid extra div wrappers:
return (
  <>
    <h1>Title</h1>
    <p>Content</p>
  </>
)

// Expressions in JSX:
return (
  <div>
    {isLoggedIn ? <Dashboard /> : <Login />}
    {items.map(i => <Item key={i.id} data={i} />)}
    {count > 0 && <Badge count={count} />}
  </div>
)
```

■ Interview Tip:
JSX is NOT HTML. One common gotcha: JSX uses className and htmlFor. Also, you can't use statements (if/for) directly in JSX — use ternary or && for conditions, .map() for lists.

Q5 What is the Virtual DOM?

■ Answer

Virtual DOM (VDOM) is a lightweight JS object tree mirroring the real DOM.
React uses it to compute minimal real DOM changes.

The 3-step process:

- 1. State changes → new VDOM tree created (fast, in-memory)
- 2. Diffing — old VDOM vs new VDOM (reconciliation algorithm)
- 3. Patching — only changed nodes updated in real DOM

Why it helps:

- Real DOM operations are expensive (cause reflow/repaint)
- Batching updates — multiple state changes = one DOM update
- Cross-browser abstraction via Synthetic Events

```
// React element (VDOM node) — just a plain JS object:
const vdom = {
  type: 'div',
  props: {
    className: 'card',
    children: [
      { type: 'h2', props: { children: 'Kaushal' } },
      { type: 'p',  props: { children: 'Developer' } }
    ]
  }
}

// State change — only 'Developer' → 'Founder' changes
// Old VDOM: { type:'p', props:{ children:'Developer' } }
// New VDOM: { type:'p', props:{ children:'Founder' } }
// Real DOM update: p.textContent = 'Founder' (1 operation!)
```

■ Interview Tip:
VDOM isn't always faster than direct DOM manipulation — React's real advantage is developer experience plus good-enough performance. For extreme performance, use canvas or WebAssembly.

Q6 What is Reconciliation?

■ Answer

Reconciliation is React's diffing algorithm that determines minimal DOM updates.

Key heuristics React uses:

- Different element type → destroy tree, create new
- Same element type → update attributes only
- List items use key prop for efficient reordering

React Fiber (React 16+):

- New reconciler that makes work interruptible
- Can pause, resume, and prioritize rendering work
- Enables Concurrent Mode features (Suspense, useTransition)

```
// Same type → update props only
// Before: <p className='old'>Text</p>
// After:  <p className='new'>Text</p>
// Only className attribute updated (minimal!)

// Different type → full rebuild
// Before: <div>hello</div>
// After:  <span>hello</span>
// div destroyed, span created from scratch

// List keys — CRITICAL for performance
// ■ Bad: index as key
items.map((item, i) => <Item key={i} data={item} />)
// Reorder items → React confuses which is which!

// ■ Good: stable unique ID
items.map(item => <Item key={item.id} data={item} />)
// React correctly matches items, only moves DOM nodes
```

■ **Interview Tip:**
The key prop is one of the most important performance and correctness tools. Never use array index as key for dynamic lists (where items can be added/removed/reordered). Always use stable unique IDs.

Q7 What is key prop? Why is it important?

■ Answer

key is a special React prop that helps identify which list items have changed.
It helps React's reconciliation algorithm be efficient and correct.

How key works:

- React uses key to match Virtual DOM nodes between renders
- Same key = same component instance (state preserved!)
- Different key = component is destroyed and recreated

Rules:

- Keys must be unique among SIBLINGS (not globally)
- Keys should be STABLE (same across renders)
- Never use index as key if list can change

Key as a reset trick:

- Changing key forces full re-mount of a component
- Use this to reset component state!

```
// ■ Index as key — causes bugs
const [items, setItems] = useState(['A', 'B', 'C'])
// Render: [<li key=0>A, <li key=1>B, <li key=2>C]
// Delete A: [<li key=0>B, <li key=1>C]
// React sees: key=0 CHANGED A→B (wrong!), not 'A was removed'
// Bug: component state from A gets applied to B!

// ■ Stable ID as key
const items = [{ id: 'a1', name:'A' }, { id:'b2', name:'B' }]
items.map(i => <Item key={i.id} data={i} />)  // correct!

// Key as reset trick:
function UserProfile({ userId }) {
  return <ProfileForm key={userId} userId={userId} />
  // When userId changes → ProfileForm fully resets!
  // All local state cleared automatically
}
```

■ Interview Tip:
The key-as-reset trick is a powerful but underused pattern. Instead of manually resetting component state in useEffect, just change the key and React will unmount+remount the component automatically.

Q8 What are Props? Props vs State?

■ Answer

Props (properties) are read-only data passed from parent to child component.
State is mutable data managed WITHIN a component.

Props:

- Passed from parent to child (like function arguments)
- Read-only — child CANNOT modify props
- Changes to parent's state → new props to child → re-render
- Can be any JS value: string, number, object, function, JSX

State:

- Local data managed inside a component
- Mutable via setState/dispatch
- Changing state triggers re-render of that component + children
- Private to the component

Decision: Props or State?

- Data comes from parent → Props
- Data is managed here, changes UI → State
- Data doesn't change → neither (plain variable)

```jsx
// Props — parent passes data to child
function Button({ label, onClick, disabled = false, variant = 'primary' }) {
  return (
    <button
      onClick={onClick}
      disabled={disabled}
      className={`btn btn-${variant}`}
    >
      {label}
    </button>
  )
}

// State — component manages its own data
function Toggle() {
  const [isOn, setIsOn] = useState(false)  // state!
  return (
    <Button
      label={isOn ? 'ON' : 'OFF'}         // prop from state
      onClick={() => setIsOn(v => !v)}     // prop = callback
      variant={isOn ? 'success' : 'danger'} // prop from state
    />
  )
}
```

■ Interview Tip:
Think of props as function arguments and state as local variables. Components can receive many props but should only manage state they truly 'own'. Everything else should be lifted up or come from a store.

Q9 What is State? What is setState?

■ Answer

State is data that when changed, causes React to re-render the component.
useState returns [state, setState] for functional components.

setState behavior:

- Schedules a re-render (asynchronous!)
- Does NOT immediately update state variable
- React batches multiple setState calls (React 18)
- Passing function (prev => newVal) uses latest state

State is snapshot-based:

- In a single render, state value is fixed (snapshot)
- useState gives you the state from THAT render
- Closures capture the snapshot value

Rules for state:

- Never mutate state directly — always create new values
- Objects/arrays must be replaced, not mutated

```jsx
function Counter() {
  const [count, setCount] = useState(0)

  // ■ Stale closure — all three use SAME count snapshot
  const badTripleInc = () => {
    setCount(count + 1)  // count=0 in this render
    setCount(count + 1)  // count still 0!
    setCount(count + 1)  // count still 0! → only +1 total
  }

  // ■ Functional updates — always use latest
  const goodTripleInc = () => {
    setCount(p => p + 1)  // 0→1
    setCount(p => p + 1)  // 1→2
    setCount(p => p + 1)  // 2→3  → +3 total ■
  }

  // ■ Mutate state directly — no re-render!
  const bad = () => { count++ }  // won't work

  // ■ Mutate object state
  const [user, setUser] = useState({ name: 'Kaushal' })
  const bad2 = () => { user.name = 'Singh' }  // won't re-render!

  // ■ Replace with new object
  const good = () => setUser(prev => ({ ...prev, name: 'Singh' }))
}
```

■ Interview Tip:
The most common React bug: calling setState multiple times and expecting each to see the previous result. Use functional updates (prev => newVal) whenever your new state depends on the old state.

Q10 What is lifting state up?

■ Answer

Lifting state up means moving state to the closest common ancestor of components that need to share it.

When to lift state:

- Two sibling components need same data
- Child needs to update parent's data
- Multiple components need to stay in sync

How to lift state:

1. Remove state from child component
2. Move state to parent
3. Pass state down as props
4. Pass setter function as callback prop

Trade-off:

- More prop passing (may cause prop drilling)
- Solution: Context API or state management library

```jsx
// Before lifting: each has own state (not synced)
function TempCelsius() {
  const [temp, setTemp] = useState(0)  // own state
  return <input value={temp} onChange={e => setTemp(e.target.value)} />
}
function TempFahrenheit() {
  const [temp, setTemp] = useState(32)  // separate state!
  return <p>Fahrenheit: {temp}</p>
}

// After lifting: parent owns shared state
function TemperatureConverter() {
  const [celsius, setCelsius] = useState(0)  // lifted to parent
  const fahrenheit = celsius * 9/5 + 32

  return (
    <div>
      <CelsiusInput celsius={celsius} onChangeCelsius={setCelsius} />
      <p>Fahrenheit: {fahrenheit}</p>  {/* derived from shared state */}
    </div>
  )
}

function CelsiusInput({ celsius, onChangeCelsius }) {
  return (
    <input value={celsius}
      onChange={e => onChangeCelsius(Number(e.target.value))} />
  )
}
```

Q11 Controlled vs Uncontrolled components

■ Answer

These describe two different ways of handling form inputs in React.

Controlled Components (React-driven):

- Input value comes from React state
- onChange updates state → state updates input (round-trip)
- React is the 'single source of truth'
- Validation, conditional submit, formatted input are easy
- More code but more control

Uncontrolled Components (DOM-driven):

- DOM manages own value, React reads via ref when needed
- Less re-renders (good for large forms)
- Simpler code for basic forms
- React Hook Form uses uncontrolled inputs for performance

Recommendation:

- Use React Hook Form (uncontrolled) for forms with many fields
- Use controlled for real-time validation or complex logic

```jsx
// Controlled component — React owns the value
function ControlledInput() {
  const [name, setName] = useState('')
  const [error, setError] = useState('')

  const handleChange = (e) => {
    const val = e.target.value
    setName(val)
    setError(val.length < 2 ? 'Too short' : '')  // validate on change!
  }

  return (
    <div>
      <input value={name} onChange={handleChange} />
      {error && <span className='error'>{error}</span>}
      <button disabled={!!error}>Submit</button>
    </div>
  )
}

// Uncontrolled component — DOM owns the value
function UncontrolledInput() {
  const inputRef = useRef(null)

  const handleSubmit = () => {
    const value = inputRef.current.value  // read on demand
    submitForm({ name: value })
  }

  return (
    <div>
```

Q12 What is conditional rendering?

■ Answer

Conditional rendering means showing different UI based on conditions.
Since JSX is just JavaScript, use JS expressions for conditions.

4 patterns for conditional rendering:

- Ternary: condition ? <A /> : <B />
- AND shortcircuit: condition && <A />
- Early return: return null to render nothing
- Element variable: store JSX in variable

```jsx
function UserCard({ user, isLoading, error }) {
  // Pattern 1: Early return for loading/error
  if (isLoading) return <Spinner />
  if (error) return <ErrorMessage msg={error.message} />
  if (!user) return null  // render nothing

  // Pattern 2: Ternary — A or B
  const badge = user.isAdmin
    ? <AdminBadge />
    : <UserBadge />

  return (
    <div className='card'>
      <h2>{user.name}</h2>

      {/* Pattern 3: && — show or nothing */}
      {user.isPremium && <PremiumBadge />}

      {/* ■■ 0 && renders '0' — use !! or ternary */}
      {user.posts.length > 0 && <PostList posts={user.posts} />}
      {/* NOT: {user.posts.length && ...} ← renders '0' when empty! */}

      {badge}  {/* Pattern 4: element variable */}
    </div>
  )
}
```

■ Interview Tip:
Common gotcha: 0 && <Component /> renders the number '0' on screen, not nothing! Always compare to ensure it's a boolean: count > 0 && <Component /> or Boolean(count) && <Component />.

Q13 What is list rendering? Why are keys important?

■ Answer

List rendering uses .map() to transform arrays of data into arrays of JSX elements.

Best practices for list rendering:

- Always add key prop to root element in map
- Use stable unique ID, not array index (if list can change)
- Index as key is OK for static, never-reordered lists
- Key must be unique among siblings, not globally
- Key doesn't get passed as prop — use a different prop if needed

```jsx
// Basic list rendering
function TodoList({ todos }) {
  return (
    <ul>
      {todos.map(todo => (
        <li key={todo.id}>  {/* key on root returned element */}
          <input type='checkbox' checked={todo.done}
            onChange={() => toggleTodo(todo.id)} />
          <span>{todo.text}</span>
          <button onClick={() => deleteTodo(todo.id)}>X</button>
        </li>
      ))}
    </ul>
  )
}

// Nested lists — keys only need unique within same list
function Categories({ categories }) {
  return categories.map(cat => (
    <div key={cat.id}>
      <h3>{cat.name}</h3>
      <ul>
        {cat.items.map(item => (
          <li key={item.id}>{item.name}</li>  // item.id unique in its list
        ))}
      </ul>
    </div>
  ))
}

// Complex: render a table from data
function DataTable({ columns, rows }) {
  return (
    <table>
      <thead><tr>{columns.map(col => <th key={col.id}>{col.label}</th>)}</tr></thead>
      <tbody>
        {rows.map(row => (
          <tr key={row.id}>
            {columns.map(col => <td key={col.id}>{row[col.key]}</td>)}
          </tr>
        ))}
      </tbody>
```

Q14 What are Hooks? Why were they introduced?

■ Answer

Hooks are functions that let functional components use React features (state, effects, context, refs) that were previously only in class components.

Why Hooks were introduced (React 16.8, 2019):

- Class components had complex lifecycle bugs ('this' binding issues)
- Logic couldn't be easily shared between components
- HOCs and render props caused 'wrapper hell'
- Classes confused both humans and compiler optimizations

Built-in hooks:

- State: useState, useReducer
- Effect: useEffect, useLayoutEffect, useInsertionEffect
- Context: useContext
- Ref: useRef, useImperativeHandle
- Performance: useMemo, useCallback
- Other: useId, useTransition, useDeferredValue, useSyncExternalStore

```
// Before hooks: HOC wrapper hell
// withRouter(withAuth(withTheme(withData(Component))))
// Hard to trace where props come from!

// With hooks: compose logic cleanly
function UserProfile({ userId }) {
  const { data: user, loading } = useFetch(`/api/users/${userId}`)
  const { theme } = useTheme()
  const { isAuthenticated } = useAuth()
  const windowSize = useWindowSize()

  if (!isAuthenticated) return <Redirect to='/login' />
  if (loading) return <Spinner />

  return <Profile user={user} theme={theme} />
}
// Each useX clearly shows what logic is used!

// Rules of hooks:
// 1. Only call at TOP LEVEL (not inside if/for/functions)
// 2. Only call inside React functions (not regular JS)
```

■ Interview Tip:
Hooks transformed React. Before hooks, you needed class components for state and lifecycle, and complex HOC patterns for logic reuse. Hooks let you write more readable and reusable code.

Q15 Rules of Hooks — why must hooks be top-level?

■ Answer

There are 2 rules of hooks enforced by ESLint plugin react-hooks:

Rule 1: Only call hooks at the TOP LEVEL

- Don't call inside if/else, loops, or nested functions
- Hooks must be called in same order every render

Rule 2: Only call hooks in React functions

- Functional components
- Custom hooks (functions starting with 'use')
- Not in regular JS functions, class methods, event handlers

WHY same order matters:

- React tracks hooks by their ORDER of calls in an array
- Slot 1 = first useState, Slot 2 = second useState, etc.
- If order changes (conditional hook), slots mismatch → bugs!

```
// React internally stores hooks like this (simplified):
// let hooks = []
// let currentHook = 0

// Render 1: [useState(0), useState(''), useEffect(...)]
// Render 2: [useState(0), useState(''), useEffect(...)]
// Same order every time → correct values returned

// ■ WRONG: hook inside condition
function BadComponent({ showExtra }) {
  const [count, setCount] = useState(0)  // always slot 0
  if (showExtra) {
    const [extra, setExtra] = useState('')  // conditional!
    // Render 1 (showExtra=true):  [count, extra]
    // Render 2 (showExtra=false): [count]
    // Next useState after would read WRONG slot!
  }
  const [name, setName] = useState('')  // slot moves! BUG!
}

// ■ CORRECT: condition inside hook logic
function GoodComponent({ showExtra }) {
  const [count, setCount] = useState(0)   // always slot 0
  const [extra, setExtra] = useState('')   // always slot 1
  const [name, setName] = useState('')     // always slot 2
  const displayExtra = showExtra ? extra : null  // condition in value
}
```

■ Interview Tip:

This is a very common interview question! The answer: React tracks hooks by their call order in an internal array. If you put a hook inside an if statement, its order could change between renders, corrupting the internal hook state.

Q16 What is useState — complete deep dive?

■ Answer

useState is the most fundamental hook — adds reactive state to functional components.

Key behaviors:

- Returns array [value, setter] — destructure immediately
- Setter schedules re-render with new value
- State is preserved between renders (unlike regular variables)
- Each component instance has its OWN independent state copy
- React 18: updates automatically batched even outside event handlers

```
function CompleteUseState() {
  // Basic
  const [count, setCount] = useState(0)
  const [name, setName] = useState('')

  // Object state — always spread to avoid mutation
  const [user, setUser] = useState({ name: '', age: 0, active: true })
  const updateAge = (age) => setUser(prev => ({ ...prev, age }))

  // Array state — always return new array
  const [items, setItems] = useState([])
  const addItem = (item) => setItems(prev => [...prev, item])
  const removeItem = (id) => setItems(prev => prev.filter(i => i.id !== id))
  const updateItem = (id, changes) => setItems(prev =>
    prev.map(i => i.id === id ? { ...i, ...changes } : i)
  )

  // Lazy init — function only runs ONCE on mount
  const [data] = useState(() => {
    const saved = localStorage.getItem('data')
    return saved ? JSON.parse(saved) : []
  })

  // Functional update — ALWAYS use when new state needs old state
  const increment = () => setCount(prev => prev + 1)

  // Batching (React 18) — both setState calls in ONE re-render
  const handleClick = () => {
    setCount(c => c + 1)  // batched
    setName('Singh')       // batched — only 1 re-render!
  }
}
```

■ Interview Tip:

Important: useState is a snapshot. Inside one render, calling setCount three times with the same base count only increments once. Use functional updates (prev => prev + 1) to always work with the latest value.

Q17 What is useEffect — complete deep dive?

■ Answer

useEffect synchronizes your component with external systems (APIs, DOM, subscriptions).
It runs AFTER React updates the DOM (after paint).

Dependency array controls when effect runs:

- No array: after EVERY render
- Empty []: once after FIRST render (mount)
- [dep1, dep2]: when dep1 or dep2 changes

Cleanup function:

- Return a function from useEffect
- Runs on unmount AND before next effect runs
- ALWAYS clean up: timers, subscriptions, event listeners, fetch

```
function UseEffectDeepDive({ userId, query }) {
  const [user, setUser] = useState(null)
  const [results, setResults] = useState([])

  // Mount + unmount (empty deps)
  useEffect(() => {
    document.title = 'App'
    return () => { document.title = 'Page' }  // restore on unmount
  }, [])

  // Re-run when userId changes
  useEffect(() => {
    const ctrl = new AbortController()
    fetch(`/api/users/${userId}`, { signal: ctrl.signal })
      .then(r => r.json()).then(setUser)
      .catch(e => { if (e.name !== 'AbortError') console.error(e) })
    return () => ctrl.abort()  // cancel if userId changes before done
  }, [userId])

  // Debounced search
  useEffect(() => {
    if (!query) return
    const timer = setTimeout(() => {
      fetch(`/api/search?q=${query}`).then(r => r.json()).then(setResults)
    }, 500)
    return () => clearTimeout(timer)  // cancel previous timer
  }, [query])

  // Subscription
  useEffect(() => {
    const socket = connectWebSocket()
    socket.on('update', handleUpdate)
    return () => socket.disconnect()  // MUST clean up!
  }, [])
}
```

Q18 useEffect vs useLayoutEffect

■ Answer

Both run after React updates, but at different points in the browser's lifecycle.

useEffect:

- Runs AFTER browser paints screen (non-blocking)
- User sees the update before effect runs
- 99% of use cases — data fetching, subscriptions, event listeners

useLayoutEffect:

- Runs AFTER DOM mutations but BEFORE browser paints
- Synchronous — blocks paint until effect runs
- Use for: reading DOM measurements, preventing visual flicker
- Similar to componentDidMount/componentDidUpdate

Rule of thumb:

- Start with useEffect — only switch to useLayoutEffect if you see visual flicker

```
// useEffect — after paint (standard)
useEffect(() => {
  // DOM is updated, browser painted
  // User can see the update
  fetchData().then(setData)
}, [])

// useLayoutEffect — before paint (for DOM measurements)
function Tooltip({ targetRef, text }) {
  const tooltipRef = useRef(null)

  useLayoutEffect(() => {
    // Read target DOM position BEFORE painting
    const rect = targetRef.current.getBoundingClientRect()
    const tooltip = tooltipRef.current

    // Position tooltip relative to target
    tooltip.style.top = `${rect.bottom + 8}px`
    tooltip.style.left = `${rect.left}px`
    // Position is set BEFORE user sees it — no flicker!
  })

  return <div ref={tooltipRef} className='tooltip'>{text}</div>
}

// If you use useEffect for positioning, user briefly sees
// tooltip in wrong position before it moves → FLICKER
```

■ **Interview Tip:**
Use useLayoutEffect when you need to read DOM measurements and immediately update them. A common example: measuring an element's size and adjusting another element's layout. Without useLayoutEffect, you'd see a flicker as positions update after paint.

Q19 What is useRef? useRef vs useState?

■ Answer

useRef returns a mutable ref object whose .current property persists across renders.

Two main uses:

- DOM reference: attach to DOM element to access it directly
- Mutable value: store value that persists but doesn't trigger re-render

useRef vs useState:

- useState: changing value → RE-RENDER (reactive)
- useRef: changing .current → NO re-render (non-reactive)

When to use useRef (not useState):

- Storing previous state value
- Tracking if component is mounted
- Storing interval/timeout IDs
- Accessing DOM elements imperatively

```jsx
function UseRefExamples() {
  // Use 1: DOM Reference — access DOM directly
  const inputRef = useRef(null)
  const focusInput = () => inputRef.current.focus()
  const getInputValue = () => inputRef.current.value

  // Use 2: Persist value without re-render
  const renderCount = useRef(0)  // track renders without causing re-render!
  renderCount.current++  // mutate .current directly (no setter)

  // Use 3: Previous value pattern
  const [count, setCount] = useState(0)
  const prevCount = useRef(0)
  useEffect(() => {
    prevCount.current = count  // save after render
  })
  // prevCount.current always has LAST render's count

  // Use 4: Store timer IDs (avoid stale closure)
  const timerRef = useRef(null)
  const start = () => {
    timerRef.current = setInterval(() => console.log('tick'), 1000)
  }
  const stop = () => clearInterval(timerRef.current)

  // Use 5: Track mounted state
  const isMounted = useRef(false)
  useEffect(() => {
    isMounted.current = true
    return () => { isMounted.current = false }
  }, [])

  return (
    <div>
```

Q20 What is useMemo? When to use it?

■ Answer

useMemo caches (memoizes) the result of a computation between renders.
It only recomputes when its dependencies change.

Syntax: const value = useMemo(() => compute(), [deps])

- First arg: function that returns the memoized value
- Second arg: dependency array — recompute when any dep changes

Use useMemo when:

- Expensive computation (filtering large arrays, complex math)
- Creating objects/arrays passed as props to React.memo children
- Values used as deps in other hooks (stable reference)

Do NOT use useMemo for:

- Cheap computations — overhead isn't worth it
- Primitive values (strings, numbers)
- When value changes every render anyway

```jsx
function DataDashboard({ data, filters, sortBy }) {
  // ■ Expensive: filter + sort 10000 items
  const processed = useMemo(() => {
    console.log('Processing data...')  // should run rarely
    return data
      .filter(item => filters.every(f => f(item)))
      .sort((a, b) => a[sortBy] > b[sortBy] ? 1 : -1)
  }, [data, filters, sortBy])

  // ■ Stable object reference for memoized child
  const chartConfig = useMemo(() => ({
    width: 600, height: 400,
    colors: ['#58A6FF', '#3FB950'],
    data: processed
  }), [processed])

  // ■ Don't memoize trivial computation
  const doubled = useMemo(() => count * 2, [count])  // overkill!
  // Just write: const doubled = count * 2

  // ■ Don't memoize if deps change every render
  const result = useMemo(() => compute(obj), [obj])
  // If obj is created inline in parent, it's new every render!

  return <Chart config={chartConfig} items={processed} />
}
```

■ **Interview Tip:**
Profile before optimizing! useMemo has overhead (deps comparison on every render). Only add it if you can measure a performance improvement. The React DevTools Profiler helps identify which components are slow.

Q21 What is useCallback? When to use it?

■ Answer

useCallback memoizes a function reference, returning the same function instance as long as dependencies don't change.

Syntax: const fn = useCallback(() => doSomething(), [deps])

Why function reference matters:

- Functions in JS: () => {} === () => {} is FALSE
- New function every render = new prop for child = child re-renders
- useCallback = same function ref = child can skip re-render

Use useCallback when:

- Passing function as prop to React.memo-wrapped child
- Function is a dep in another hook's dep array
- Event handlers for heavy components

useCallback vs useMemo:

- useCallback(fn, deps) = useMemo(() => fn, deps)
- useCallback = cache function | useMemo = cache value

```
// When useCallback matters:

const Parent = () => {
  const [count, setCount] = useState(0)
  const [text, setText] = useState('')

  // ■ New function every render when text changes
  // → HeavyList re-renders even though its data didn't change
  const handleItemClick = (id) => { console.log('clicked', id, count) }

  // ■ Same function ref until count changes
  const handleItemClickCached = useCallback((id) => {
    console.log('clicked', id, count)
  }, [count])  // re-create only when count changes

  return (
    <div>
      <input value={text} onChange={e => setText(e.target.value)} />
      <p>Count: {count}</p>
      {/* This re-renders on every keystroke without useCallback */}
      <HeavyList onItemClick={handleItemClickCached} />
    </div>
  )
}

// useCallback only helps if child uses React.memo!
const HeavyList = React.memo(({ onItemClick }) => {
  console.log('HeavyList rendering...')  // only when prop changes
  return <ul>{items.map(i => <li key={i.id} onClick={() => onItemClick(i.id)}>{i.name}</li>)}</ul>
})
```

Q22 What is useContext? How to use Context API?

■ Answer

useContext reads the value of a React context — it's the consumer side of Context API.
Context API solves prop drilling by creating global-like state for a subtree.

Context API — 3 parts:

- createContext(defaultValue) — creates the context
- Context.Provider value={...} — provides the value
- useContext(Context) — reads the current value

Performance caveat:

- ALL consumers re-render when context value changes
- Split contexts by update frequency
- Memoize context value with useMemo

Context vs Redux:

- Context: low-frequency updates, small-medium apps
- Redux: high-frequency updates, complex state, time-travel debug

```jsx
// 1. Create context
const AuthContext = createContext(null)

// 2. Provider — wrap your app
function AuthProvider({ children }) {
  const [user, setUser] = useState(null)
  const [token, setToken] = useState(null)

  const login = useCallback(async (credentials) => {
    const { user, token } = await authAPI.login(credentials)
    setUser(user); setToken(token)
    localStorage.setItem('token', token)
  }, [])

  const logout = useCallback(() => {
    setUser(null); setToken(null)
    localStorage.removeItem('token')
  }, [])

  const value = useMemo(() => ({
    user, token, login, logout,
    isAuthenticated: !!user
  }), [user, token, login, logout])

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>
}

// 3. Custom hook to consume (better than useContext directly)
function useAuth() {
  const ctx = useContext(AuthContext)
  if (!ctx) throw new Error('useAuth must be inside AuthProvider')
  return ctx
}
```

Q23 What are custom hooks?

■ Answer

Custom hooks are functions starting with 'use' that encapsulate reusable stateful logic.
They can call other hooks and return whatever values they need to.

Rules for custom hooks:

- Must start with 'use' (enforced by lint)
- Can call other hooks inside
- Each component using the hook gets ISOLATED state
- Sharing logic ≠ sharing state

Common patterns:

- Wrap useEffect + useState into useFetch
- Wrap complex business logic
- Abstract third-party library usage
- Provide reusable form/input logic

```
// useLocalStorage — persist state to localStorage
function useLocalStorage(key, initialValue) {
  const [storedValue, setStoredValue] = useState(() => {
    try {
      const item = window.localStorage.getItem(key)
      return item ? JSON.parse(item) : initialValue
    } catch { return initialValue }
  })

  const setValue = useCallback((value) => {
    try {
      const valueToStore = value instanceof Function ? value(storedValue) : value
      setStoredValue(valueToStore)
      window.localStorage.setItem(key, JSON.stringify(valueToStore))
    } catch(e) { console.error(e) }
  }, [key, storedValue])

  return [storedValue, setValue]
}

// useToggle — simple boolean toggle
function useToggle(initialValue = false) {
  const [value, setValue] = useState(initialValue)
  const toggle = useCallback(() => setValue(v => !v), [])
  const setTrue = useCallback(() => setValue(true), [])
  const setFalse = useCallback(() => setValue(false), [])
  return [value, { toggle, setTrue, setFalse }]
}

// useOnClickOutside — detect clicks outside element
function useOnClickOutside(ref, handler) {
  useEffect(() => {
    const listener = (e) => {
      if (!ref.current || ref.current.contains(e.target)) return
      handler(e)  // click was outside!
```

Q24 What is useReducer?

■ Answer

useReducer is an alternative to useState for managing complex state logic.
It follows the Redux pattern: actions describe what happened, reducers compute new state.

useState vs useReducer:

- useState — simple, few values, independent updates
- useReducer — complex logic, multiple related values, named transitions

Benefits of useReducer:

- All state logic in ONE place (reducer function)
- State transitions are explicit (named actions)
- Easy to test — reducer is pure function
- dispatch is stable — no need for useCallback

useReducer + Context = mini Redux

- Create context with state + dispatch
- Any component can dispatch actions
- Reducer handles all state updates centrally

```
// Shopping cart with useReducer
const ACTIONS = {
  ADD: 'ADD_ITEM',
  REMOVE: 'REMOVE_ITEM',
  UPDATE_QTY: 'UPDATE_QUANTITY',
  CLEAR: 'CLEAR_CART'
}

function cartReducer(state, action) {
  switch(action.type) {
    case ACTIONS.ADD:
      const existing = state.find(i => i.id === action.item.id)
      if (existing) {
        return state.map(i => i.id === action.item.id
          ? { ...i, qty: i.qty + 1 } : i)
      }
      return [...state, { ...action.item, qty: 1 }]

    case ACTIONS.REMOVE:
      return state.filter(i => i.id !== action.id)

    case ACTIONS.UPDATE_QTY:
      return state.map(i => i.id === action.id
        ? { ...i, qty: Math.max(1, action.qty) } : i)

    case ACTIONS.CLEAR:
      return []

    default: return state
  }
}

```

Q25 useId, useTransition, useDeferredValue — new hooks

■ Answer

React 18 introduced powerful new hooks for concurrent features.

useId():

- Generates stable unique ID for accessibility (aria-* attributes)
- Works with SSR — same ID on server and client
- Don't use for list keys!

useTransition():

- Marks state update as non-urgent (low priority)
- UI stays responsive while transition happens
- Returns [isPending, startTransition]

useDeferredValue(value):

- Creates a 'deferred' version of a value
- Deferred value lags behind to keep UI responsive
- Similar to debounce but React-native

```jsx
// useId — stable accessible IDs
function FormField({ label, type = 'text' }) {
  const id = useId()  // stable: ':r0:', ':r1:', etc.
  return (
    <div>
      <label htmlFor={id}>{label}</label>
      <input id={id} type={type} />
    </div>
  )
}

// useTransition — keep UI responsive during heavy updates
function SearchPage() {
  const [query, setQuery] = useState('')
  const [results, setResults] = useState([])
  const [isPending, startTransition] = useTransition()

  const handleSearch = (e) => {
    setQuery(e.target.value)  // urgent: update input immediately
    startTransition(() => {
      // non-urgent: this can wait
      const data = filterLargeDataset(e.target.value)
      setResults(data)  // won't block typing!
    })
  }

  return (
    <div>
      <input value={query} onChange={handleSearch} />
      {isPending && <Spinner />}
      <ResultsList results={results} />
    </div>
  )
```

Q26 What is React Fiber?

■ Answer

React Fiber is the complete rewrite of React's reconciliation engine (React 16, 2017).
It makes rendering work incremental, interruptible, and prioritizable.

Problems with old reconciler (React < 16):

- Reconciliation was synchronous and recursive
- Couldn't be paused once started
- Long renders blocked the main thread → janky UI

What Fiber enables:

- Split work into small units (fibers)
- Pause/resume/abort work between frames
- Assign priorities to different updates
- Enables Concurrent Mode features

Fiber node:

- Each React element gets a Fiber node
- Forms a linked list (not recursive tree)
- Contains: type, props, state, effect, parent/child/sibling links

Two phases:

- Render/Reconcile phase — interruptible (find what changed)
- Commit phase — synchronous (apply changes to DOM)

```
// Fiber conceptually (simplified):
const fiber = {
  type: 'div',           // element type
  props: { className: 'container' },
  stateNode: domNode,    // actual DOM node
  parent: parentFiber,   // linked list (not tree!)
  child: childFiber,
  sibling: siblingFiber,
  alternate: oldFiber,   // previous render's fiber
  effectTag: 'UPDATE',   // what to do: INSERT/UPDATE/DELETE
  expirationTime: 100,   // priority
}

// Two-phase rendering:
// Phase 1: Render (interruptible)
// Walk fiber tree, compute new state, find differences
// Can be paused if higher priority work comes in

// Phase 2: Commit (synchronous, not interruptible)
// Apply all DOM changes at once
// Fire useEffect, useLayoutEffect

// This is why useLayoutEffect fires synchronously
// and useEffect fires asynchronously after paint
```

Q27 What is React.memo? When to use it?

■ Answer

React.memo is a Higher-Order Component that memoizes a component's render.
If props are the same (shallow comparison), the component skips re-rendering.

React.memo vs useMemo vs useCallback:

- React.memo — memoize a COMPONENT (skip re-render)
- useMemo — memoize a VALUE (cache computation)
- useCallback — memoize a FUNCTION (stable reference)

When React.memo helps:

- Component renders frequently due to parent re-renders
- Component receives same props often
- Component is expensive to render

When React.memo doesn't help:

- Props change every render anyway
- Component is cheap to render
- Component renders rarely

Custom comparison:

- React.memo(Component, customEqualityFn)
- Return true = skip re-render, false = re-render

```
// Basic React.memo usage
const Avatar = React.memo(function Avatar({ user }) {
  console.log('Avatar rendering for:', user.name)
  return (
    <div className='avatar'>
      <img src={user.avatar} alt={user.name} />
      <p>{user.name}</p>
    </div>
  )
})

// ■ Won't benefit — new object prop every render
function Parent() {
  const [count, setCount] = useState(0)
  return (
    <div>
      <button onClick={() => setCount(c => c+1)}>{count}</button>
      <Avatar user={{ name: 'Kaushal', avatar: '/k.jpg' }} />
      {/* New object literal every render → Avatar ALWAYS re-renders */}
    </div>
  )
}

// ■ Works — stable user reference
function BetterParent({ userId }) {
  const [count, setCount] = useState(0)
  const user = useMemo(() => getUser(userId), [userId])  // stable!
  return (
```

Q28 What is HOC (Higher-Order Component)?

■ Answer

A HOC is a function that takes a component and returns a new enhanced component.
Pattern: const EnhancedComponent = withSomething(OriginalComponent)

What HOCs are used for:

- Cross-cutting concerns: auth, permissions, logging
- Code reuse before hooks existed
- Render props alternative

HOC rules:

- Don't mutate the original component
- Pass through unrelated props (use spread)
- Copy display name for DevTools

HOCs vs Hooks:

- Hooks replaced most HOC use cases
- HOCs create extra component nesting
- Hooks are more composable and easier to understand

When HOCs are still useful:

- Error boundaries (class component wrapping)
- React.memo (is itself a HOC!)
- react-redux connect() (legacy)

```
// withAuth HOC — protect routes
function withAuth(WrappedComponent) {
  function WithAuth(props) {
    const { isAuthenticated, user } = useAuth()

    if (!isAuthenticated) {
      return <Redirect to='/login' />
    }

    // Pass through ALL original props + injected auth prop
    return <WrappedComponent {...props} currentUser={user} />
  }

  // Copy display name for React DevTools
  WithAuth.displayName = `withAuth(${WrappedComponent.displayName || WrappedComponent.name})`
  return WithAuth
}

// Usage:
const ProtectedDashboard = withAuth(Dashboard)

// withLogger HOC — log component renders
function withLogger(WrappedComponent) {
  return function WithLogger(props) {
    useEffect(() => {
      console.log(`${WrappedComponent.name} mounted with`, props)
      return () => console.log(`${WrappedComponent.name} unmounted`)
```

Q29 What are Portals in React?

■ Answer

Portals let you render a child component into a different DOM node than the parent component's DOM hierarchy.

Why Portals?

- Modal dialogs need to be at document body level (CSS stacking context)
- Tooltips that overflow their container
- Notifications/toasts that overlay everything

How to create a portal:

ReactDOM.createPortal(child, container)

- child: React element to render
- container: DOM node to render into

Important behavior:

- Events still bubble through React tree (not DOM tree)
- Portal still behaves as React child for context, state, etc.

```
import ReactDOM from 'react-dom'

// Modal component using Portal
function Modal({ isOpen, onClose, children, title }) {
  if (!isOpen) return null

  return ReactDOM.createPortal(
    <div className='modal-overlay' onClick={onClose}>
      <div className='modal-box' onClick={e => e.stopPropagation()}>
        <div className='modal-header'>
          <h2>{title}</h2>
          <button onClick={onClose}>X</button>
        </div>
        <div className='modal-body'>{children}</div>
      </div>
    </div>,
    document.getElementById('modal-root')  // different DOM node!
  )
}

// HTML needs a portal container:
// <body>
//   <div id='root'></div>
//   <div id='modal-root'></div>  ← portal renders here
// </body>

// Usage — still gets context from parent!
function App() {
  const [showModal, setShowModal] = useState(false)
  const theme = useTheme()  // context works across portal!

  return (
    <div className='app'>
```

Q30 What is Error Boundary?

■ Answer

Error Boundaries catch JavaScript errors in child component tree, log them, and display a fallback UI instead of crashing the whole app.

Important limitations:

- Only class components can be Error Boundaries
- No hook equivalent yet (but React team is working on it)
- Catches: render errors, lifecycle methods, constructor
- Does NOT catch: event handlers, async code, SSR, errors in boundary itself

Lifecycle methods:

- static getDerivedStateFromError(error) — update state to show fallback
- componentDidCatch(error, info) — log to error service

Best practices:

- Wrap separate sections (not entire app)
- Use react-error-boundary library for easier usage

```jsx
// Error Boundary class component
class ErrorBoundary extends React.Component {
  state = { hasError: false, error: null }

  static getDerivedStateFromError(error) {
    // Called after error is thrown, before re-render
    return { hasError: true, error }
  }

  componentDidCatch(error, errorInfo) {
    // Log to error tracking service
    console.error('Error caught:', error)
    Sentry.captureException(error, { extra: errorInfo })
  }

  render() {
    if (this.state.hasError) {
      return this.props.fallback || (
        <div className='error-page'>
          <h2>Something went wrong</h2>
          <button onClick={() => this.setState({ hasError: false })}>
            Try Again
          </button>
        </div>
      )
    }
    return this.props.children
  }
}

// Usage — wrap different sections
function App() {
  return (
```

Q31 What is React.lazy and Suspense?

■ Answer

React.lazy enables code splitting — loading components on demand instead of upfront.
Suspense shows a fallback while lazy-loaded components are loading.

Why code splitting?

- Reduce initial bundle size
- Load code only when needed
- Faster initial page load

How it works:

- React.lazy(() => import('./Component')) — dynamic import
- Component is fetched when first rendered
- Suspense shows fallback during loading

What Suspense handles:

- React.lazy (code splitting)
- Data fetching (React 18 with Suspense-enabled libraries)
- Server Components streaming

```jsx
import { lazy, Suspense } from 'react'
import { Routes, Route } from 'react-router-dom'

// Lazy load route-level components
const Dashboard = lazy(() => import('./pages/Dashboard'))
const UserProfile = lazy(() => import('./pages/UserProfile'))
const Analytics = lazy(() => import('./pages/Analytics'))

// Suspense provides fallback during loading
function AppRoutes() {
  return (
    <Suspense fallback={<PageSpinner />}>
      <Routes>
        <Route path='/' element={<Home />} />  {/* loaded upfront */}
        <Route path='/dashboard' element={<Dashboard />} />  {/* lazy */}
        <Route path='/users/:id' element={<UserProfile />} />  {/* lazy */}
        <Route path='/analytics' element={<Analytics />} />  {/* lazy */}
      </Routes>
    </Suspense>
  )
}

// Nested Suspense — finer loading UI control
function Dashboard() {
  const Sidebar = lazy(() => import('./Sidebar'))
  const Chart = lazy(() => import('./Chart'))

  return (
    <div className='dashboard'>
      <Suspense fallback={<SidebarSkeleton />}>
        <Sidebar />
      </Suspense>
```

Q32 CSR vs SSR vs SSG — what are they?

■ Answer

Three rendering strategies with different trade-offs for performance and SEO.

CSR — Client-Side Rendering:

- Empty HTML shell, JS downloads and renders everything
- Fast after initial load, slow first paint
- Bad for SEO (crawler sees empty page)
- Good for: admin panels, dashboards, apps behind login

SSR — Server-Side Rendering:

- Server renders HTML for each request
- User sees content immediately (good FCP)
- Great for SEO — HTML has content on first load
- More server compute, can be slower TTFB
- Good for: e-commerce product pages, news, social feeds

SSG — Static Site Generation:

- HTML generated at BUILD TIME
- Blazing fast — just serve static files from CDN
- Perfect SEO, no server needed
- Bad for frequently changing data
- Good for: blogs, docs, marketing pages, portfolios

```
// CSR (Create React App / Vite — pure client)
// index.html: <div id='root'></div>  (empty!)
// App.js downloads, renders everything in browser

// SSR with Next.js — getServerSideProps
// pages/product/[id].js
export async function getServerSideProps({ params }) {
  // Runs on server for EVERY request
  const product = await db.products.findById(params.id)
  return { props: { product } }  // sent to component
}

export default function ProductPage({ product }) {
  return <div><h1>{product.name}</h1><p>■{product.price}</p></div>
}

// SSG with Next.js — getStaticProps
// pages/blog/[slug].js
export async function getStaticPaths() {
  const posts = await getAllPosts()
  return {
    paths: posts.map(p => ({ params: { slug: p.slug } })),
    fallback: false
  }
}

export async function getStaticProps({ params }) {
  // Runs at BUILD TIME only
```

Q33 What is Redux? Why use it?

■ Answer

Redux is a predictable state container for JavaScript apps.
It stores the entire app state in a single JS object.

3 Core Principles:

- Single source of truth — one store for all state
- State is read-only — only actions can change state
- Changes via pure functions — reducers are pure functions

Redux data flow:

- UI dispatches action → reducer computes new state → store updates → UI re-renders

When to use Redux:

- Large apps with complex shared state
- State needs to be accessible from many components
- Complex state transitions
- Need time-travel debugging / state persistence

When NOT to use Redux:

- Small/medium apps (Context + useReducer is enough)
- Mostly local component state
- Team is not familiar with Redux patterns

```
// Redux data flow: Action → Reducer → Store → UI

// 1. Action — plain object describing what happened
const addTodo = { type: 'todos/add', payload: { id: 1, text: 'Learn Redux' } }

// 2. Reducer — pure function: (oldState, action) => newState
function todosReducer(state = [], action) {
  switch(action.type) {
    case 'todos/add': return [...state, action.payload]
    case 'todos/remove': return state.filter(t => t.id !== action.payload)
    case 'todos/toggle': return state.map(t =>
      t.id === action.payload ? { ...t, done: !t.done } : t)
    default: return state
  }
}

// 3. Store — holds state, handles dispatch
import { createStore } from 'redux'
const store = createStore(todosReducer)

store.getState()  // [] — current state
store.dispatch(addTodo)  // dispatch action
store.subscribe(() => renderUI())  // react to changes

// 4. React component — connect via react-redux
function TodoList() {
  const todos = useSelector(state => state.todos)
  const dispatch = useDispatch()
```

Q34 What is Redux Toolkit (RTK)? createSlice?

■ Answer

Redux Toolkit is the official, modern way to write Redux code.
It eliminates boilerplate and adds common utilities.

What RTK includes:

- configureStore — simplified store setup with DevTools
- createSlice — generates actions + reducer from one definition
- createAsyncThunk — handles async operations cleanly
- createEntityAdapter — normalized state management
- RTK Query — powerful data fetching + caching

createSlice magic:

- Automatically creates action creators from reducer names
- Uses Immer internally — write 'mutations' that are actually immutable
- Less code = fewer bugs

```
import { configureStore, createSlice } from '@reduxjs/toolkit'

// createSlice — actions + reducer in ONE place
const cartSlice = createSlice({
  name: 'cart',
  initialState: { items: [], total: 0, itemCount: 0 },

  reducers: {
    // Immer lets you write 'mutations' (actually immutable behind scenes!)
    addItem(state, action) {
      const existing = state.items.find(i => i.id === action.payload.id)
      if (existing) {
        existing.qty += 1  // looks like mutation — Immer handles it!
      } else {
        state.items.push({ ...action.payload, qty: 1 })
      }
      state.total += action.payload.price
      state.itemCount += 1
    },

    removeItem(state, action) {
      const idx = state.items.findIndex(i => i.id === action.payload)
      if (idx !== -1) {
        state.total -= state.items[idx].price * state.items[idx].qty
        state.itemCount -= state.items[idx].qty
        state.items.splice(idx, 1)  // Immer handles immutability!
      }
    },

    clearCart(state) {
      state.items = []; state.total = 0; state.itemCount = 0
    }
  }
})

```

Q35 What is createAsyncThunk?

■ Answer

createAsyncThunk is RTK's utility for handling async operations (API calls) in Redux.
It automatically dispatches pending/fulfilled/rejected actions.

What it generates:

- thunkName/pending — dispatched before async starts
- thunkName/fulfilled — dispatched on success
- thunkName/rejected — dispatched on failure

Handle in extraReducers:

- builder.addCase(thunk.pending, reducer)
- builder.addCase(thunk.fulfilled, reducer)
- builder.addCase(thunk.rejected, reducer)

Error handling:

- Errors are caught automatically
- Access in rejected via action.error.message
- Use rejectWithValue for custom error payloads

```
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit'

// Define async thunk
export const fetchUsers = createAsyncThunk(
  'users/fetchAll',
  async (params, { rejectWithValue }) => {
    try {
      const res = await fetch(`/api/users?page=${params.page}`)
      if (!res.ok) return rejectWithValue({ status: res.status })
      return await res.json()  // becomes action.payload
    } catch(err) {
      return rejectWithValue(err.message)
    }
  }
)

export const createUser = createAsyncThunk(
  'users/create',
  async (userData) => {
    const res = await fetch('/api/users', {
      method: 'POST',
      body: JSON.stringify(userData),
      headers: { 'Content-Type': 'application/json' }
    })
    return res.json()
  }
)

// Handle in slice
const usersSlice = createSlice({
  name: 'users',
  initialState: { list: [], loading: false, error: null },
```

Q36 What is RTK Query?

■ Answer

RTK Query is a powerful data fetching and caching tool built into Redux Toolkit.
It eliminates the need to write async thunks and loading/error state manually.

What RTK Query provides:

- Automatic caching with configurable invalidation
- Loading and error state out of the box
- Automatic refetching when data is stale
- Polling, optimistic updates, infinite queries
- Auto-generated hooks

RTK Query vs React Query:

- RTK Query — integrated with Redux store
- React Query — standalone, no Redux needed
- Both solve the same problem well
- Use RTK Query if already using Redux, React Query otherwise

```
import { createApi, fetchBaseQuery } from '@reduxjs/toolkit/query/react'

// Define API
export const userApi = createApi({
  reducerPath: 'userApi',
  baseQuery: fetchBaseQuery({
    baseUrl: '/api',
    prepareHeaders: (headers, { getState }) => {
      const token = getState().auth.token
      if (token) headers.set('Authorization', `Bearer ${token}`)
      return headers
    }
  }),
  tagTypes: ['User'],  // for cache invalidation

  endpoints: (builder) => ({
    // Query endpoint (GET)
    getUsers: builder.query({
      query: (page = 1) => `/users?page=${page}`,
      providesTags: ['User']  // this data is tagged 'User'
    }),

    getUserById: builder.query({
      query: (id) => `/users/${id}`,
      providesTags: (result, error, id) => [{ type: 'User', id }]
    }),

    // Mutation endpoint (POST/PUT/DELETE)
    createUser: builder.mutation({
      query: (userData) => ({
        url: '/users', method: 'POST', body: userData
      }),
      invalidatesTags: ['User']  // invalidate cache → refetch users!
    }),
```

Q37 Redux vs Context API — when to use which?

■ Answer

Both manage shared state, but they're optimized for different use cases.

Context API:

- Built into React, no extra library
- Good for: theme, auth, locale, user preferences
- Low-frequency updates (theme doesn't change every second)
- Simple data shapes
- All consumers re-render on any value change

Redux (RTK):

- Separate library, more setup
- Good for: complex app state, high-frequency updates
- useSelector subscribes to only relevant slice of state
- Time-travel debugging, DevTools
- Middleware for side effects
- Better performance for large apps

Rule of thumb:

- Small/medium app: Context + useReducer is enough
- Large app with complex state: RTK
- Auth/theme: Context
- Cart/products/orders: Redux or RTK Query

```
// Context: good for auth (low frequency change)
const AuthContext = createContext(null)
// Changes maybe once per session login/logout

// ■ Context for high-frequency updates (performance issue)
const CartContext = createContext(null)
// Updating cart on every add/remove → ALL consumers re-render
// With 100 components using CartContext → 100 re-renders!

// ■ Redux for cart: only subscribed components re-render
const CartTotal = () => {
  // Only re-renders when cart.total changes!
  const total = useSelector(state => state.cart.total)
  return <p>Total: ■{total}</p>
}

const ItemCount = () => {
  // Only re-renders when cart.itemCount changes!
  const count = useSelector(state => state.cart.itemCount)
  return <Badge count={count} />
}

// Performance difference with useSelector:
// Even if entire cart state changes,
// CartTotal only re-renders if cart.total changed
// This is the key advantage of Redux's selector system
```

Q38 How to optimize a React app's performance?

■ Answer

Performance optimization in React should be measured first, then applied.

Key optimization techniques:

- React.memo — skip component re-renders when props same
- useMemo — cache expensive computed values
- useCallback — stable function references for memoized children
- Code splitting — React.lazy + Suspense for route/component level
- Virtualization — render only visible items (react-window)
- Avoid anonymous functions in JSX
- useTransition — mark updates as non-urgent
- Bundle analysis — webpack-bundle-analyzer, identify large deps

```
// 1. Code splitting at route level (biggest win!)
const Dashboard = lazy(() => import('./Dashboard'))
const Analytics = lazy(() => import('./Analytics'))

// 2. Virtualize long lists
import { FixedSizeList } from 'react-window'
function VirtualList({ items }) {
  const Row = ({ index, style }) => (
    <div style={style}><UserRow user={items[index]} /></div>
  )
  return (
    <FixedSizeList height={600} itemCount={items.length} itemSize={72}>
      {Row}
    </FixedSizeList>
  )
}  // renders only ~10 items even if items has 10000!

// 3. Avoid new references in render
// ■ New array every render → child always re-renders
return <Chart colors={['red', 'blue']} />
// ■ Stable reference
const colors = useMemo(() => ['red', 'blue'], [])
return <Chart colors={colors} />

// 4. useTransition for search
const [query, setQuery] = useState('')
const [isPending, startTransition] = useTransition()
const handleChange = e => {
  setQuery(e.target.value)  // urgent
  startTransition(() => setResults(filter(e.target.value)))  // non-urgent
}

// 5. Correct key usage — stable IDs prevent DOM recreation
todos.map(t => <Todo key={t.id} {...t} />)  // not key={index}
```

■ **Interview Tip:**
Always profile BEFORE optimizing! Use React DevTools Profiler to find the actual bottleneck. The most impactful wins are usually:

Q39 How to handle forms in React at scale?

■ Answer

For production forms, use React Hook Form or Formik — don't reinvent the wheel.

Problems with manual controlled forms:

- Re-renders on EVERY keystroke (all fields in one state object)
- Validation logic scattered
- Form state management boilerplate

React Hook Form (recommended):

- Uncontrolled inputs by default = no re-render on keystroke
- Only re-renders on submit or specific watchers
- Built-in validation with resolver support (Zod, Yup)
- 80% less code than manual form handling

React Hook Form vs Formik:

- RHF: 30x fewer re-renders, smaller bundle, better performance
- Formik: older, more tutorials, wider adoption historically
- RHF is the clear winner in 2024

```jsx
import { useForm } from 'react-hook-form'
import { zodResolver } from '@hookform/resolvers/zod'
import { z } from 'zod'

// Schema with Zod
const schema = z.object({
  name: z.string().min(2, 'Min 2 chars').max(50),
  email: z.string().email('Invalid email'),
  age: z.number().min(18, 'Must be 18+').max(100),
  password: z.string().min(8, 'Min 8 chars')
    .regex(/[A-Z]/, 'Need uppercase')
    .regex(/[0-9]/, 'Need number'),
})

function SignupForm({ onSuccess }) {
  const {
    register, handleSubmit, formState: { errors, isSubmitting },
    reset, setError
  } = useForm({ resolver: zodResolver(schema) })

  const onSubmit = async (data) => {
    try {
      await createUser(data)
      reset()  // clear form after success
      onSuccess()
    } catch(err) {
      setError('email', { message: 'Email already taken' })
    }
  }

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
```

Q40 How to handle authentication in React?

■ Answer

Authentication in React involves: storing token, protecting routes, managing auth state.

Complete auth flow:

1. User logs in → receive JWT token
2. Store token securely (HttpOnly cookie preferred over localStorage)
3. Include token in API requests
4. Check auth state to protect routes
5. Handle token expiry + refresh tokens

Token storage options:

- localStorage — easy but XSS vulnerable (avoid for JWTs)
- HttpOnly Cookie — secure, set by server, XSS can't read
- Memory (state) — safest, lost on refresh

Route protection:

- Protected route component that checks auth state
- Redirect to login if not authenticated

```jsx
// AuthContext + Protected Routes
const AuthContext = createContext(null)

function AuthProvider({ children }) {
  const [user, setUser] = useState(null)
  const [loading, setLoading] = useState(true)

  // Check existing session on mount
  useEffect(() => {
    authAPI.getMe()  // verify token from httpOnly cookie
      .then(user => setUser(user))
      .catch(() => setUser(null))
      .finally(() => setLoading(false))
  }, [])

  const login = async (credentials) => {
    const { user } = await authAPI.login(credentials)
    // server sets httpOnly cookie automatically
    setUser(user)
  }

  const logout = async () => {
    await authAPI.logout()  // server clears cookie
    setUser(null)
  }

  if (loading) return <AppSpinner />
  return <AuthContext.Provider value={{ user, login, logout, isAuthenticated: !!user }}>
    {children}</AuthContext.Provider>
}

// Protected Route component
function ProtectedRoute({ children }) {
```

Q41 How to implement infinite scrolling in React?

■ Answer

Infinite scrolling loads more data automatically as the user scrolls to the bottom.

Implementation approaches:

- IntersectionObserver — observe a sentinel element at the bottom
- Scroll event — less efficient, fires too often
- react-infinite-scroll-component library
- Virtualized lists (react-window) for very large datasets

Key state needed:

- data — accumulated items from all pages
- page — current page number
- hasMore — whether more data exists
- loading — prevent double-fetching

```
function useInfiniteScroll(fetchFn) {
  const [items, setItems] = useState([])
  const [page, setPage] = useState(1)
  const [hasMore, setHasMore] = useState(true)
  const [loading, setLoading] = useState(false)
  const loaderRef = useRef(null)  // sentinel element

  // Fetch when page changes
  useEffect(() => {
    setLoading(true)
    fetchFn(page).then(({ data, total }) => {
      setItems(prev => [...prev, ...data])
      setHasMore(items.length + data.length < total)
      setLoading(false)
    })
  }, [page])

  // Observe sentinel element
  useEffect(() => {
    const observer = new IntersectionObserver(([entry]) => {
      if (entry.isIntersecting && hasMore && !loading) {
        setPage(prev => prev + 1)  // load next page!
      }
    }, { threshold: 0.1 })

    if (loaderRef.current) observer.observe(loaderRef.current)
    return () => observer.disconnect()
  }, [hasMore, loading])

  return { items, loading, hasMore, loaderRef }
}

// Usage:
function PostFeed() {
  const { items, loading, hasMore, loaderRef } = useInfiniteScroll(
    (page) => fetch(`/api/posts?page=${page}`).then(r => r.json())
```

Q42 How to handle debounced search in React?

■ Answer

Debounced search delays API calls until the user stops typing.
This avoids spamming the API with every keystroke.

Implementation:

- Input state updates immediately (responsive UI)
- API call delayed by 500ms after last keystroke
- Cancel previous timer on each keystroke

3 approaches:

- Custom useDebounce hook
- useCallback + useEffect
- Lodash debounce with useRef

Also consider:

- AbortController to cancel in-flight requests
- Show loading state during debounce + fetch

```
// Approach 1: useDebounce custom hook (cleanest)
function useDebounce(value, delay) {
  const [debounced, setDebounced] = useState(value)
  useEffect(() => {
    const timer = setTimeout(() => setDebounced(value), delay)
    return () => clearTimeout(timer)  // cancel on value change
  }, [value, delay])
  return debounced
}

function SearchBox() {
  const [query, setQuery] = useState('')
  const [results, setResults] = useState([])
  const [isLoading, setIsLoading] = useState(false)

  const debouncedQuery = useDebounce(query, 500)

  useEffect(() => {
    if (!debouncedQuery.trim()) { setResults([]); return }

    const ctrl = new AbortController()
    setIsLoading(true)

    fetch(`/api/search?q=${debouncedQuery}`, { signal: ctrl.signal })
      .then(r => r.json())
      .then(data => { setResults(data); setIsLoading(false) })
      .catch(e => { if (e.name !== 'AbortError') setIsLoading(false) })

    return () => ctrl.abort()
  }, [debouncedQuery])

  return (
    <div>
```

Q43 What is React Server Components?

■ Answer

React Server Components (RSC) is a new paradigm where components run ONLY on the server.
Introduced in React 18, used by default in Next.js 13+ App Router.

Client vs Server Components:

- Server Components (default in Next.js App Router)
- Render on server, send HTML to client
- Can directly access DB, files, APIs (no fetch overhead!)
- Zero JS bundle cost — no component code sent to browser
- No hooks, no event handlers, no browser APIs
- Client Components ('use client' directive)
- Run in browser, have interactivity
- Can use useState, useEffect, event handlers
- Traditional React components

Benefits:

- Dramatically reduce JS bundle size
- Automatic code splitting
- Direct database access without API layer

```
// Server Component (Next.js App Router)
// app/users/page.js
// NO 'use client' = server component by default

async function UsersPage() {
  // Can directly query DB! No API route needed!
  const users = await db.users.findMany()  // runs on server

  return (
    <div>
      <h1>Users</h1>
      {users.map(user => (
        <UserCard key={user.id} user={user} />
      ))}
    </div>
  )
}

// Client Component — for interactivity
'use client'

function SearchBar({ onSearch }) {
  const [query, setQuery] = useState('')  // useState needs client

  return (
    <input
      value={query}
      onChange={e => { setQuery(e.target.value); onSearch(e.target.value) }}
    />
  )
}
```

Q44 How to structure a scalable React application?

■ Answer

Folder structure should be feature-based (colocation) for large apps.

Feature-based structure (recommended):

- Group by feature/domain, not by file type
- All files for a feature live together
- Easy to find related code
- Easy to delete a feature

Common structure:

- src/features/ — feature modules
- src/components/ — shared UI components
- src/hooks/ — shared custom hooks
- src/services/ — API/data layer
- src/store/ — Redux store
- src/utils/ — utility functions

Barrel files (index.js):

- Export everything from a folder's index.js
- Clean imports: import { UserCard } from 'features/users'

```
// Scalable React project structure:
// src/
//   features/
//     users/
//       components/
//         UserCard.jsx
//         UserList.jsx
//         UserForm.jsx
//       hooks/
//         useUsers.js
//         useUserSearch.js
//       store/
//         usersSlice.js
//         usersSelectors.js
//       pages/
//         UsersPage.jsx
//         UserDetailPage.jsx
//       api/
//         usersAPI.js
//       index.js  ← barrel file
//
//     products/  (same structure)
//     orders/
//     auth/
//
//   components/  (shared UI)
//     Button/
//     Modal/
//     DataTable/
//
```

Q45 React interview: Explain your architecture choices for a large app

■ Answer

This is an open-ended senior question about your decision-making process.

Framework: Next.js

- SSR/SSG for SEO, performance, and SEO
- App Router for RSC + layouts
- Built-in API routes, image optimization

State Management Strategy:

- Server state: RTK Query or React Query (caching, refetching)
- Global UI state: Redux Toolkit (cart, auth)
- Form state: React Hook Form + Zod
- Local component state: useState/useReducer

Performance Strategy:

- Code splitting at route level with lazy()
- React.memo + useCallback for expensive trees
- Virtualization for large lists
- Image optimization (next/image)

Testing Strategy:

- Unit: Vitest/Jest for utilities and reducers
- Component: React Testing Library
- E2E: Playwright/Cypress for critical flows

```
// Architecture decision examples:

// 1. Server state vs UI state separation
// Server state (from API): React Query / RTK Query
const { data: users } = useGetUsersQuery()
// Client UI state: useState/Redux
const [selectedUserId, setSelectedUserId] = useState(null)

// 2. Type safety throughout
// Zod schema → TypeScript types → React Hook Form → API validation
const schema = z.object({ name: z.string().min(2), email: z.string().email() })
type FormData = z.infer<typeof schema>  // TypeScript type from schema

// 3. Centralized error handling
// Global error boundary at app level + feature boundaries
// API error handling in axios interceptor
axiosInstance.interceptors.response.use(
  response => response,
  error => {
    if (error.response?.status === 401) dispatch(logout())
    if (error.response?.status >= 500) showGlobalError(error)
    return Promise.reject(error)
  }
)

// 4. Feature flags for gradual rollout
```

Q46 What is the difference between controlled and uncontrolled forms? Which is better?

■ Answer

Controlled: React state is single source of truth for input value.
Uncontrolled: DOM manages the value, React reads via ref when needed.

Controlled pros:

- Real-time validation, conditional rendering based on input
- Programmatic control (reset, pre-fill, format)
- Easier to test (state in React)

Uncontrolled pros:

- Fewer re-renders (no state update on keystroke)
- Simpler code for basic forms
- How React Hook Form works internally

Recommendation:

- Use React Hook Form (uncontrolled) for most forms
- Use controlled only when you need real-time UI changes
- Never manually implement large controlled forms from scratch

```
// React Hook Form — uncontrolled but feature-rich
const { register, handleSubmit, formState: { errors } } = useForm()

// register() hooks into the DOM input directly (uncontrolled)
<input {...register('email', { required: 'Email required' })} />
{errors.email && <span>{errors.email.message}</span>}
```

■ Interview Tip:
In interviews: mention React Hook Form uses uncontrolled inputs for performance. A form with 20 fields won't re-render the entire form on every keystroke — only the validation messages update.

Q47 Axios vs Fetch API — which to use?

■ Answer

Both make HTTP requests, but Axios is more feature-rich.

Fetch (built-in):

- No installation needed
- Promise-based
- Must manually check res.ok for errors
- Manual JSON parsing: await res.json()
- No request cancellation (need AbortController)

Axios (library):

- Automatic JSON parsing
- Throws error for 4xx/5xx automatically
- Interceptors (auth headers, error handling globally)
- Request cancellation built-in
- Request/response transformation

Recommendation:

- Small app: Fetch is enough
- Large app: Axios for interceptors and global error handling

```
// Fetch — manual error handling
const res = await fetch('/api/users')
if (!res.ok) throw new Error(`HTTP ${res.status}`)  // manual!
const data = await res.json()  // manual parsing

// Axios — automatic
const { data } = await axios.get('/api/users')  // throws on 4xx/5xx

// Axios interceptor — global auth + error handling
axios.defaults.baseURL = 'https://api.app.com'
axios.interceptors.request.use(config => {
  config.headers.Authorization = `Bearer ${getToken()}`
  return config
})
axios.interceptors.response.use(
  res => res,
  err => {
    if (err.response?.status === 401) logout()
    return Promise.reject(err)
  }
)
```

■ **Interview Tip:**

For production apps, Axios interceptors are invaluable. Instead of adding auth headers in every request, you add them once in an interceptor. Same for error handling — centralize it instead of try/catch everywhere.

Q48 What is prop drilling and how to avoid it?

■ Answer

Prop drilling is passing props through multiple intermediate components that don't need them, just to get data to a deeply nested component.

Solutions in order of preference:

- Component composition — restructure to avoid drilling
- Context API — for truly global data (auth, theme)
- State management — Redux/Zustand for complex shared state

Component composition trick:

- Pass the component itself as a prop (children pattern)
- Parent renders deep component, passes it to intermediary
- Intermediary doesn't need to know about the data

```jsx
// ■ Prop drilling — UserRole passes through Header, Nav
function App() {
  const [userRole, setUserRole] = useState('admin')
  return <Header userRole={userRole} />  // passes through
}
function Header({ userRole }) {
  return <Nav userRole={userRole} />  // passes through!
}
function Nav({ userRole }) {
  return <AdminLink visible={userRole === 'admin'} />  // finally used
}

// ■ Composition — pass component, not data
function App() {
  const [userRole, setUserRole] = useState('admin')
  return (
    <Header>
      {userRole === 'admin' && <AdminLink />}
    </Header>
  )
}
function Header({ children }) {
  return <nav>{children}</nav>  // no need to know about userRole!
}

// ■ Context — for truly global data
const { userRole } = useAuthContext()  // access anywhere
```

■ Interview Tip:
Component composition is the underused solution to prop drilling. Before reaching for Context, try passing the component as children or as a prop. This works when the parent knows what to render.

Q49 What is Zustand? When would you use it over Redux?

■ Answer

Zustand is a minimal, fast state management library — much simpler than Redux.

Zustand vs Redux:

- Zustand: minimal API, no boilerplate, no actions/reducers
- Redux: more structure, DevTools, time-travel debug, large ecosystem

When to choose Zustand:

- Medium-complexity shared state
- Team wants simplicity over structure
- Don't need Redux DevTools or time-travel debug
- Small-medium apps

When to choose Redux:

- Large teams who need enforced patterns
- Complex state machines with many transitions
- Need Redux DevTools time-travel debugging
- Already using Redux ecosystem

```
import { create } from 'zustand'

// Create store — no actions, no reducers, no boilerplate!
const useCartStore = create((set, get) => ({
  items: [],
  total: 0,

  addItem: (item) => set(state => ({
    items: [...state.items, item],
    total: state.total + item.price
  })),

  removeItem: (id) => set(state => {
    const item = state.items.find(i => i.id === id)
    return {
      items: state.items.filter(i => i.id !== id),
      total: state.total - (item?.price ?? 0)
    }
  }),

  getItemCount: () => get().items.length  // selector function
}))

// Use in any component — NO Provider needed!
function CartIcon() {
  const itemCount = useCartStore(state => state.items.length)
  return <Badge count={itemCount} />
}

function Cart() {
  const { items, total, removeItem } = useCartStore()
  return (
```

Q50 What is React Query? Why use it?

■ Answer

React Query (TanStack Query) is a powerful server state management library.
It handles caching, background refetching, and synchronization with server state.

What React Query solves:

- Caching API responses — same data not fetched twice
- Automatic background refetching when window refocuses
- Loading and error states out of the box
- Optimistic updates — UI updates before server confirms
- Infinite queries, pagination

Server State vs Client State:

- Client state: UI state (modal open, selected tab) → useState/Redux
- Server state: data from API → React Query
- This distinction is React Query's core value proposition

```jsx
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query'

// Fetch and cache data
function UserProfile({ userId }) {
  const { data: user, isLoading, error } = useQuery({
    queryKey: ['user', userId],  // cache key — unique per user
    queryFn: () => fetch(`/api/users/${userId}`).then(r => r.json()),
    staleTime: 5 * 60 * 1000,  // consider fresh for 5 minutes
    refetchOnWindowFocus: true   // refetch when user returns to tab
  })

  if (isLoading) return <Skeleton />
  if (error) return <ErrorMessage />
  return <ProfileCard user={user} />
}

// Mutation with cache invalidation
function EditProfile({ userId }) {
  const queryClient = useQueryClient()

  const mutation = useMutation({
    mutationFn: (data) => fetch(`/api/users/${userId}`, {
      method: 'PUT', body: JSON.stringify(data)
    }).then(r => r.json()),

    onSuccess: (updatedUser) => {
      // Update cache directly
      queryClient.setQueryData(['user', userId], updatedUser)
      // OR: invalidate and refetch
      // queryClient.invalidateQueries({ queryKey: ['user', userId] })
    }
  })

  return <form onSubmit={d => mutation.mutate(d)}>...</form>
}
```

Q51 How does React handle synthetic events?

■ Answer

React wraps browser native events in SyntheticEvent — a cross-browser wrapper.

Why Synthetic Events:

- Browser APIs are inconsistent (IE vs Chrome vs Firefox)
- SyntheticEvent provides consistent API across browsers
- Old React pooled events (recycled objects) — React 17 removed this

React 17 change:

- Events now attached to root element, not window/document
- Improves interoperability with non-React code
- Event pooling removed

Synthetic Event methods:

- e.preventDefault() — stop default browser behavior
- e.stopPropagation() — stop event bubbling
- e.target — actual DOM element that triggered event
- e.currentTarget — element event listener is attached to
- e.nativeEvent — access original browser event

```jsx
function EventDemo() {
  const handleClick = (e) => {
    // e is SyntheticEvent
    e.preventDefault()    // stop default
    e.stopPropagation()  // stop bubbling

    console.log(e.target)         // element clicked
    console.log(e.currentTarget)  // element with handler
    console.log(e.nativeEvent)    // original browser event
    console.log(e.type)           // 'click'
  }

  // Form submit
  const handleSubmit = (e) => {
    e.preventDefault()  // MUST prevent form's default page reload
    const formData = new FormData(e.target)
    submitForm(Object.fromEntries(formData))
  }

  // Input change
  const handleChange = (e) => {
    const { name, value, type, checked } = e.target
    const fieldValue = type === 'checkbox' ? checked : value
    setForm(prev => ({ ...prev, [name]: fieldValue }))
  }

  return (
    <form onSubmit={handleSubmit}>
      <input name='username' onChange={handleChange} />
      <input name='remember' type='checkbox' onChange={handleChange} />
      <button onClick={handleClick}>Submit</button>
```

Q52 What is forwardRef and useImperativeHandle?

■ Answer

forwardRef allows a parent component to pass a ref down to a child's DOM node.
useImperativeHandle customizes what the parent sees when using the ref.

When to use forwardRef:

- Library components that need to expose a DOM node
- Input components that need to be focused programmatically
- Animation libraries that need direct DOM access

useImperativeHandle:

- Instead of exposing the whole DOM node, expose specific methods
- Creates a controlled imperative API for the component
- Hides internal implementation details

Rule:

- Use forwardRef sparingly — declarative props are usually better
- Use when you genuinely need imperative operations

```jsx
import { forwardRef, useImperativeHandle, useRef } from 'react'

// forwardRef — pass ref to internal DOM node
const CustomInput = forwardRef(function CustomInput({ label, ...props }, ref) {
  return (
    <div className='field'>
      <label>{label}</label>
      <input ref={ref} {...props} />  {/* forward ref to input */}
    </div>
  )
})

// Parent can focus the input directly:
function Form() {
  const emailRef = useRef(null)
  const focusEmail = () => emailRef.current.focus()
  return (
    <div>
      <CustomInput ref={emailRef} label='Email' type='email' />
      <button onClick={focusEmail}>Focus Email</button>
    </div>
  )
}

// useImperativeHandle — expose custom API
const VideoPlayer = forwardRef(function VideoPlayer(props, ref) {
  const videoRef = useRef(null)

  useImperativeHandle(ref, () => ({
    // Only expose these methods, not the whole DOM node!
    play: () => videoRef.current.play(),
    pause: () => videoRef.current.pause(),
    seek: (time) => { videoRef.current.currentTime = time }
```

Q53 Complete React interview final tips

■ Answer

Final set of React concepts commonly asked in interviews:

React 18 key features:

- Automatic batching — setState calls batched even in async code
- Concurrent features — Suspense, useTransition, useDeferredValue
- createRoot — new API for rendering (instead of ReactDOM.render)
- Server Components — zero-bundle components (Next.js App Router)

Common interview mistakes to avoid:

- Mutating state directly (user.name = 'x') — always spread!
- Missing useEffect deps (causes stale closure bugs)
- Using index as list key (causes state bugs on reorder)
- No cleanup in useEffect (causes memory leaks)
- Overusing useContext (re-renders all consumers)

Must-know interview topics:

- Reconciliation + keys
- Closure in hooks (stale closure bug)
- When to use which state solution
- Performance optimization techniques
- SSR/CSR/SSG trade-offs

```
// React 18 automatic batching
// React < 18: batching only in event handlers
setTimeout(() => {
  setCount(c => c+1)  // React 17: 2 re-renders
  setUser(u => ({...u, active: true}))
}, 0)

// React 18: automatic batching EVERYWHERE
setTimeout(() => {
  setCount(c => c+1)     // Batched!
  setUser(u => ({...u})) // Only 1 re-render now ■
}, 0)

// createRoot (React 18 API)
import { createRoot } from 'react-dom/client'
const root = createRoot(document.getElementById('root'))
root.render(<App />)  // enables concurrent features

// Stale closure bug — common interview question
function StaleExample() {
  const [count, setCount] = useState(0)
  useEffect(() => {
    const timer = setInterval(() => {
      console.log(count)  // STALE — always logs 0!
      // count is captured from closure at mount time
    }, 1000)
    return () => clearInterval(timer)
```

React + Redux — Complete! ■

You just mastered 53 React + Redux questions

Next up: Node.js → Express → MongoDB → Auth → System Design

■ SAVE this post

■■ REPOST to help your team

■ COMMENT 'JS950' for full PDF

■ FOLLOW Kaushal Singh

Kaushal Singh
Full-Stack Dev | MERN + React Native | Built Fintech Apps | Kanmotech Pvt. Ltd.
1M+ Impressions | Follow for daily React · JS · Interview Prep
■■ Part 2/10 done | Next: Node.js 130 Q | Comment 'JS950' for everything